CMR UNIVERSITY
Private University Estd in Karnataka State by Act No. 45 of 2013

SCHOOL OF ENGINEERING AND TECHNOLOGY

MINI PROJECT

on

"Hybrid Retrieval-Augmented Generation System for Intelligent Document Question Answering Using Local Large Language Models"

For the requirement of 2nd Semester

(4PGPR4011 – Mini Project)

II Semester

M.Tech in Artificial Intelligence

Submitted By

Name	USN
SAGAR G	**(25BMTAI015)**

Submitted to

Dr S Elango
Associate Professor,
Dept. of CSE(AI&ML),
CMR University, Bengaluru

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(AI&ML)

CMR University

Off Hennur - Bagalur Main Road,

Near Kempegowda International Airport, Chagalahatti,

Bengaluru, Karnataka-562149

Academic Year: 2025-26

CMR UNIVERSITY

Private University Estd in Karnataka State by Act No. 45 of 2013

SCHOOL OF ENGINEERING AND TECHNOLOGY

Chagalahatti, Bengaluru, Karnataka-562149

Department of Computer Science and Engineering (AIML)

CERTIFICATE

Certified that the Mini Project Work entitled "**Hybrid Retrieval-Augmented Generation System for Intelligent Document Question Answering Using Local Large Language Models"** carried out by **SAGAR G (25BMTAI015)** bonafide students of **School of Engineering and Technology**, in partial fulfillment for the award of **Master of Technology** in 2nd Semester Artificial Intelligence of CMR University, Bengaluru during the academic year 2025-2026. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report. The mini project has been approved as it satisfies the academic requirements in respect of 4PGPR4011 – Mini Project.

Signature of the guide	Signature of the HOD	Signature of the Dean
……	……	……
(Dr S Elango)	(Dr. Bhagavant K Deshpande)	(Dr. Kannan N)
Dept. of CSE(AI&ML) SoET, CMRU, Bangalore	Dept. of CSE(AI&ML) SoET, CMRU, Bangalore	SoET, CMRU, Bangalore

Name of the Examiners:	Signature with Date:
1. ……	……
2. ……	……

DECLARATION

I, **SAGAR G (25BMTAI015)** students of MASTER OF TECHNOLOGY, ARTIFICIAL INTELLIGENCE, CMR University, Bengaluru, hereby declare that the Mini Project Work entitled **"Hybrid Retrieval-Augmented Generation System for Intelligent Document Question Answering Using Local Large Language Models"** submitted by me, for the award of the Master's degree in Artificial Intelligence is a record of bonafide work carried out independently by me under the supervision and guidance of Dr s Elango, Associate Professor, Dept of CSE(AI&ML), CMR University. I hereby declare that the mini project work submitted by me is original and has not been submitted or used previously, either in whole or in part, for the award of any other degree, diploma, or other academic qualification at this University or any other institute or University.

Place: Bengaluru SAGAR G

Date: 11/07/2026 (25BMTAI015)

ACKNOWLEDGEMENT

The satisfaction of successfully completing this project would be incomplete without expressing my sincere gratitude to the people whose constant guidance, encouragement, and support made this work possible. Without their invaluable assistance, my efforts would not have been successful.

I express my sincere gratitude to my mini project guide **Dr S Elango, Associate Professor, Dept of CSE(AI&ML)**, School of Engineering and Technology, CMR University, Bangalore, without whose constant guidance and support the project would not be successful.

I express my sincere gratitude to the Mini Project Coordinator, **Dr. Prabhakar K, Assistant Professor**, Department of Computer Science and Engineering (AI&ML), School of Engineering and Technology, CMR University, Bangalore, for his constant guidance, encouragement, and support throughout the course of this project.

I would like to express my thanks **to Dr. Bhagavant K Deshpande, Professor and Head, Department of Computer Science and Engineering (AI&ML)**, School of Engineering and Technology, CMR University, Bangalore, for his encouragement that motivated me for the successful completion of Project work.

I express my heartfelt sincere gratitude **to Dr. Kannan N, Dean (Lakeside Campus), Director-School of Engineering and Technology**, CMR University for his support.

SAGAR G

ABSTRACT

The rapid growth of digital documents across organizations, educational institutions, healthcare, and enterprise environments has created a significant demand for intelligent document retrieval and question answering systems. Traditional keyword-based search techniques often fail to understand the semantic meaning of user queries, while cloud-based Large Language Models (LLMs) introduce privacy and security concerns by requiring sensitive documents to be uploaded to external servers. This project presents a Hybrid Retrieval-Augmented Generation (RAG) System for Intelligent Document Question Answering using Local Large Language Models. The proposed system integrates semantic retrieval using the BGE-M3 embedding model, keyword-based retrieval through Hybrid Search, FlashRank for reranking, and a locally deployed Llama 3.2 model executed using llama-cpp-python. The entire retrieval pipeline is managed using the RAGLite framework, enabling efficient document ingestion, chunking, embedding generation, retrieval, and context-aware response generation. The application is implemented using Python and Streamlit, providing an interactive interface for uploading PDF documents and querying them in natural language. By performing all processing locally, the proposed system ensures enhanced privacy, offline accessibility, and reduced dependency on cloud-based AI services. The developed framework demonstrates the effectiveness of combining semantic retrieval, keyword search, reranking, and local language models to deliver accurate, reliable, and context-aware answers from uploaded documents while maintaining data confidentiality.

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	Overall System Architecture	10
3.2	Use Case Model of the Proposed System	13
3.3	System Workflow Activity Diagram	13
5.1	Home Page of the Hybrid RAG Application	27
5.2	PDF Upload Interface	27
5.3	Document Processing Status	28
5.4	Model Configuration Screen	28
5.5	Question Input Interface	29
5.6	Generated Response	29

LIST OF TABLES

Table No.	Title	Page No.
2.1	Comparative Analysis of Existing Document Retrieval Techniques	6
3.1	Functional Requirements	12
3.2	Non-Functional Requirements	12
3.3	Dataset Schema	14
3.4	Document Chunk and Embedding Information	14
4.1	Software Requirements	19
4.2	Python Libraries	19
4.3	Hardware Requirements	20
4.4	Technologies Used in the Project	20
6.1	System Performance Evaluation	30

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	Declaration	ii
	Acknowledgement	iii
	Abstract	iv
	List of Figures	v
	List of Tables	vi
1	**Introduction**	1
1.1	Background of the Study	1
1.2	Problem Statement	1
1.3	Objectives of the Project	2
1.4	Scope of the Project	2
1.5	Organization of the Report	2
2	**Literature Review**	4
2.1	Review of Existing Systems	4
2.2	Comparative Analysis	6
2.3	Research Gap Identification	7
3	**System Analysis and Design**	9
3.1	Proposed System	9
3.2	System Architecture	10
3.3	Functional Requirements	11
3.4	Non-Functional Requirements	12
3.5	Use Case Diagram	12
3.6	Activity Diagram	13
3.7	Database Design	14
4	**Methodology**	16
4.1	Development Approach	16

4.2	Algorithms / Techniques Used	17
4.3	Tools and Technologies	19
5	**Implementation**	22
5.1	Module Description	22
5.2	User Interface Design	24
5.3	Coding and Integration	24
5.4	Screenshots and Outputs	26
6	**Results and Discussion**	30
6.1	Experimental Results	30
6.2	System Performance Analysis	31
6.3	Discussion of Results	32
7	**Conclusion and Future Scope**	34
7.1	Conclusion	34
7.2	Future Enhancements	35
	References	36
	Appendices	37
	Source Code Snippets	40

CHAPTER I

INTRODUCTION

1.1 Background of the Study

The rapid increase in digital documents has created a need for intelligent systems that can quickly retrieve relevant information from large collections of files. PDF documents are widely used because they preserve formatting and can easily be shared across different platforms. However, finding specific information within lengthy documents remains a difficult task.

Traditional search systems rely on keyword matching, where results are returned only if the searched words exactly match the words present in the document. This approach often fails when users ask questions using different words or phrases with similar meanings. Semantic search improves this limitation by understanding the meaning of the text, but it may sometimes overlook important keywords or technical terms.

Retrieval-Augmented Generation (RAG) has emerged as an effective solution for document question answering. Instead of depending only on the knowledge stored in a language model, RAG retrieves relevant information from uploaded documents and provides it as context before generating an answer. This improves the factual accuracy of responses and reduces the possibility of incorrect or hallucinated information.

The proposed project builds upon this concept by implementing a Hybrid Retrieval-Augmented Generation pipeline using RAGLite, semantic embeddings, keyword retrieval, FlashRank reranking, and a locally deployed Llama 3.2 model to provide intelligent and privacy-preserving document question answering.

1.2 Problem Statement

Organizations, educational institutions, and businesses manage a large amount of information in PDF documents. Retrieving useful information from these documents using traditional search methods is often difficult because keyword-based search cannot understand the contextual meaning of user queries. Cloud-based AI systems provide better language understanding but require internet connectivity and may expose sensitive documents to external servers, creating privacy and security concerns.

Semantic search alone also has limitations because it may fail to identify exact technical keywords or domain-specific terminology. Therefore, there is a need for an intelligent document question

answering system that combines semantic understanding with keyword matching while ensuring offline accessibility and data privacy.

The proposed Hybrid Retrieval-Augmented Generation system addresses these challenges by integrating semantic retrieval, keyword retrieval, reranking, and a Local Large Language Model to provide accurate and context-aware answers from uploaded PDF documents.

1.3 Objectives of the Project

The main objectives of the proposed project are:

- To develop an intelligent document question answering system using Retrieval-Augmented Generation (RAG).
- To implement Hybrid Search by combining semantic retrieval and keyword-based retrieval.
- To process uploaded PDF documents through text extraction, chunking, and embedding generation.
- To improve retrieval accuracy using FlashRank reranking.
- To integrate a Local Large Language Model for context-aware response generation.
- To develop a privacy-preserving system that operates without depending on cloud-based AI services.
- To provide a simple and user-friendly interface using Streamlit.

1.4 Scope of the Project

The scope of this project is to develop an intelligent document question answering system capable of processing PDF documents and generating accurate responses based on their content. The system allows users to upload one or more PDF files, after which the documents are processed into text chunks and converted into vector embeddings. Hybrid Search combines semantic retrieval and keyword retrieval to identify relevant document sections, while FlashRank improves the ranking of retrieved information before it is provided to the Local Large Language Model for response generation.

The system can be applied in educational institutions, research organizations, businesses, and industries where large collections of digital documents need to be searched efficiently. Since all processing is performed locally, the system improves data privacy and allows offline document

question answering. Future enhancements may include OCR support for scanned documents, multilingual capabilities, voice-based interaction, and integration with scalable vector databases.

1.5 Organization of the Report

This report is organized into seven chapters to present the proposed system in a systematic manner.

- **Chapter 1** introduces the project, background, problem statement, objectives, scope, and report organization.
- **Chapter 2** presents the literature review of recent research related to Retrieval-Augmented Generation, document retrieval, and Local Large Language Models.
- **Chapter 3** describes the system analysis and design, including the proposed architecture and system requirements.
- **Chapter 4** explains the methodology, implementation steps, algorithms, and technologies used in the project.
- **Chapter 5** presents the implementation details and screenshots of the developed application.
- **Chapter 6** discusses the results obtained from the implemented system along with performance analysis.
- **Chapter 7** concludes the project and suggests possible future enhancements.

CHAPTER II

LITERATURE REVIEW

2.1 Review of Existing Systems

With the rapid growth of Artificial Intelligence (AI) and Natural Language Processing (NLP), intelligent document retrieval and question answering systems have gained significant attention. Organizations, educational institutions, and businesses generate a large number of digital documents such as research papers, reports, manuals, and policies. Retrieving useful information from these documents manually is time-consuming and inefficient. To overcome this problem, researchers have proposed several document retrieval and question answering techniques based on keyword search, semantic search, Retrieval-Augmented Generation (RAG), and Large Language Models (LLMs).

Earlier document retrieval systems mainly relied on keyword matching techniques. Recent advancements in NLP have introduced semantic retrieval methods capable of understanding the contextual meaning of user queries. More recently, Retrieval-Augmented Generation (RAG) has combined document retrieval with Large Language Models to provide more accurate and context-aware responses. The following sections discuss the evolution of document retrieval techniques along with their advantages and limitations.

2.1.1 Traditional Keyword-Based Search

Traditional document retrieval systems primarily use keyword matching techniques to locate information within documents. These systems search for exact words or phrases entered by the user and return documents containing those keywords.

Common keyword search techniques include:

- Exact keyword matching
- Boolean search
- Pattern matching
- Full-text search

Keyword-based search methods are simple to implement and computationally efficient. However, they cannot understand the semantic meaning of a user's query. If different words are used to describe the same concept, these systems often fail to retrieve the required information. As the size of document collections increases, keyword-only search becomes less effective for intelligent question answering.

2.1.2 Semantic Search

Semantic search improves traditional document retrieval by understanding the meaning and context of words rather than relying only on exact keyword matching. Instead of comparing words directly, semantic search converts both documents and user queries into vector embeddings using embedding models.

These vector representations capture the semantic relationship between different words and sentences. As a result, semantic search can retrieve relevant information even when the user's question uses different wording from the document.

Although semantic search significantly improves contextual understanding, it may sometimes overlook exact technical keywords, abbreviations, or domain-specific terms that are important for accurate document retrieval.

2.1.3 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) combines document retrieval with Large Language Models to improve question answering accuracy. Instead of relying only on the knowledge stored within the language model, the system first retrieves relevant information from uploaded documents and then provides that information as context before generating the final response.

The RAG approach reduces hallucination, improves factual accuracy, and enables the language model to answer questions related to user-provided documents without retraining the model. It has become one of the most widely adopted techniques for intelligent document question answering applications.

However, the performance of a RAG system depends heavily on the quality of document retrieval. If irrelevant information is retrieved, the generated response may also become less accurate.

2.1.4 Local Large Language Models

Large Language Models have demonstrated remarkable capabilities in understanding and generating human language. Most commercially available LLMs operate through cloud-based APIs, requiring internet connectivity and external processing of user data.

Local Large Language Models provide an alternative approach by running the language model directly on the user's computer. Models such as Llama 3.2 can perform inference locally using optimized formats like GGUF together with inference engines such as llama-cpp-python.

Running LLMs locally improves data privacy, reduces dependency on cloud services, and enables offline document question answering. However, local inference requires sufficient hardware resources and efficient model optimization to achieve acceptable performance.

2.1.5 Hybrid Retrieval Systems

Recent research focuses on Hybrid Retrieval systems that combine multiple retrieval techniques to improve search accuracy. Instead of relying only on semantic search or keyword search, Hybrid Retrieval combines both approaches to utilize their individual strengths.

A typical Hybrid Retrieval framework integrates:

- Semantic retrieval using vector embeddings
- Keyword-based retrieval
- Reranking of retrieved documents
- Large Language Model for response generation

The reranking stage further improves retrieval quality by selecting the most relevant document chunks before passing them to the language model. This approach significantly enhances contextual understanding while maintaining support for exact keyword matching.

The proposed project adopts this Hybrid Retrieval approach by combining semantic retrieval, keyword retrieval, FlashRank-based reranking, and a locally deployed Llama 3.2 model using the RAGLite framework. This implementation provides accurate, privacy-preserving, and offline document question answering.

2.2 Comparative Analysis

Table 2.1 compares commonly used document retrieval and question answering techniques based on contextual understanding, keyword support, retrieval accuracy, privacy, and overall limitations.

Table 2.1 Comparative Analysis of Existing Document Retrieval Techniques

Method	Semantic Understanding	Keyword Matching	Privacy	Context-Aware Answers	Limitations
Keyword-Based Search	Manual	Low	High	Moderate	Assumes feature independence

Semantic Search	Manual	Low	High	High for small datasets	Requires handcrafted features
Cloud-Based LLM	Manual	Low	Low	High	Poor contextual representation
Traditional RAG	Automatic	Moderate	Modetate	High	Cannot fully utilize bidirectional context
Hybrid RAG (Proposed)	Very High	High	High	Yes	Slightly higher computational complexity

Comparative Discussion

Traditional keyword-based search systems perform well when exact search terms are available but fail to understand the contextual meaning of user queries. Semantic search improves retrieval by identifying conceptually similar information but may overlook important keywords or technical terms.

Cloud-based Large Language Models provide powerful natural language understanding but introduce privacy concerns because user documents are processed on external servers. Traditional RAG systems improve response accuracy by retrieving relevant document information before generation; however, retrieval performance depends on the quality of the search mechanism.

The proposed Hybrid Retrieval-Augmented Generation framework combines semantic retrieval, keyword retrieval, reranking, and Local Large Language Models to provide more accurate, privacy-preserving, and context-aware document question answering.

2.3 Research Gap Identification

Although significant progress has been made in document retrieval and question answering systems, several challenges remain unresolved.

Research Gap 1: Limited Semantic Understanding in Traditional Search

Traditional keyword-based search systems rely on exact word matching and cannot effectively understand the contextual meaning of user queries.

Research Gap 2: Privacy Concerns in Cloud-Based AI Systems

Most existing document question answering systems depend on cloud-based Large Language Models, requiring users to upload confidential documents to external servers.

Research Gap 3: Limited Retrieval Accuracy

Semantic retrieval alone may fail to identify exact technical keywords, while keyword search cannot capture contextual relationships. Using either method independently often reduces retrieval accuracy.

Research Gap 4: Inadequate Context Selection

Many retrieval systems provide document chunks directly to the language model without reranking them based on relevance. This can reduce the quality of generated responses.

Research Gap 5: Offline Document Question Answering

Several existing AI-based document assistants require continuous internet connectivity and cloud services, limiting their use in privacy-sensitive or offline environments.

Motivation for the Proposed Work

The proposed project addresses these research gaps by implementing a Hybrid Retrieval-Augmented Generation framework that integrates:

- Semantic retrieval using BGE-M3 embedding vectors.
- Keyword-based retrieval for exact term matching.
- FlashRank reranking to improve retrieval relevance.
- Local Llama 3.2 Large Language Model for offline response generation.
- RAGLite framework for document processing, retrieval, and response generation.
- Streamlit-based user interface for interactive document question answering.

By combining these technologies, the proposed system provides accurate, context-aware, privacy-preserving, and offline document question answering while overcoming many limitations of existing document retrieval systems.

CHAPTER III

SYSTEM ANALYSIS AND DESIGN

3.1 Proposed System

The proposed system is a Hybrid Retrieval-Augmented Generation (RAG) System for Intelligent Document Question Answering using Local Large Language Models. The system is designed to enable users to upload PDF documents and ask questions in natural language. Unlike traditional document search systems that rely only on keyword matching, the proposed system combines semantic retrieval and keyword-based retrieval to improve the accuracy of information retrieval.

The application is developed using the RAGLite framework, which manages document processing, retrieval, and response generation. After a PDF document is uploaded through the Streamlit interface, the system extracts the textual content and divides it into smaller overlapping text chunks. Each chunk is converted into vector embeddings using the BGE-M3 embedding model, enabling semantic comparison between the document content and user queries.

When a user submits a question, the query is also converted into an embedding vector. The system performs Hybrid Search, which combines semantic similarity search and keyword matching to retrieve the most relevant document chunks. These retrieved chunks are further processed using FlashRank, which reranks the results according to their relevance.

Finally, the selected document context is passed to the Llama 3.2 Large Language Model, which is executed locally using llama-cpp-python. The language model generates a context-aware response, which is displayed to the user through the Streamlit interface. Since the complete pipeline operates locally, the system provides improved privacy, offline accessibility, and reduced dependency on cloud-based AI services.

Workflow of the Proposed System

The overall workflow of the proposed system consists of the following stages:

1. PDF Upload
2. Text Extraction
3. Text Chunking
4. Embedding Generation
5. Local Document Indexing
6. User Query Processing

7. Hybrid Search (Semantic + Keyword)
8. FlashRank Reranking
9. Context Construction
10. Local LLM Response Generation
11. Display Generated Answer

Advantages of the Proposed System

The proposed Hybrid RAG framework offers several advantages over conventional document retrieval systems:

- Provides intelligent document question answering using natural language.
- Combines semantic retrieval and keyword search to improve retrieval accuracy.
- Generates context-aware responses using a Local Large Language Model.
- Ensures user privacy by processing documents locally.
- Supports offline document question answering without relying on cloud-based AI services.
- Offers a modular architecture that can be extended with additional AI models and document formats.

3.2 System Architecture

The proposed architecture consists of multiple interconnected modules that process uploaded PDF documents and generate intelligent responses based on user queries.

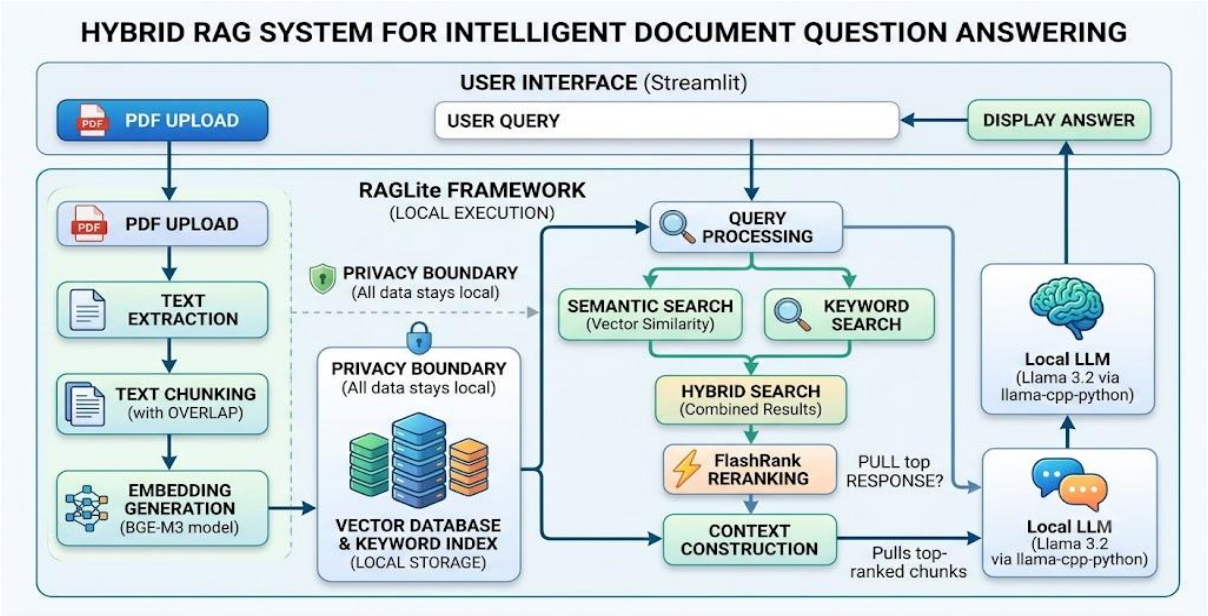


Figure 3.1 Overall System Architecture

Architectural Components

• User Interface

Developed using Streamlit, it allows users to upload PDF documents, configure models, and submit questions.

• Document Processing Module

Extracts textual information from uploaded PDF documents and prepares it for further processing.

• Text Chunking Module

Splits the extracted document into smaller overlapping chunks to improve retrieval efficiency and preserve contextual information.

• Embedding Generation Module

Uses the BGE-M3 embedding model to convert document chunks and user queries into dense vector embeddings representing their semantic meaning.

• Hybrid Retrieval Module

Performs both semantic retrieval and keyword-based retrieval to identify the most relevant document chunks.

• FlashRank Reranking Module

Ranks the retrieved document chunks according to their relevance before passing them to the language model.

• Local Large Language Model

Uses the Llama 3.2 GGUF model running through llama-cpp-python to generate accurate and context-aware responses.

• Output Module

Displays the generated answer through the Streamlit interface.

3.3 Functional Requirements

Functional requirements describe the primary operations that the proposed system must perform.

Table 3.1 Functional Requirements

Requirement ID	Functional Requirement	Description
FR1	PDF Upload	Upload PDF documents into the system
FR2	Text Extraction	Extract textual content from uploaded PDFs
FR3	Text Chunking	Divide documents into smaller text chunks
FR4	Embedding Generation	Generate vector embeddings using BGE-M3
FR5	Hybrid Search	Retrieve relevant document chunks
FR6	FlashRank Reranking	Rank retrieved chunks based on relevance
FR7	Local LLM Inference	Generate responses using Llama 3.2
FR8	Display Response	Display generated answers to the user
FR9	Document Management	Process multiple uploaded PDF documents

3.4 Non-Functional Requirements

The system should satisfy the following quality attributes.

Table 3.2 Non-Functional Requirements

Category	Requirement
Performance	Fast document retrieval and response generation
Reliability	Consistent response generation using retrieved context
Scalability	Ability to process multiple PDF documents
Maintainability	Modular implementation for future enhancements
Security	Local processing to preserve document privacy
Usability	User-friendly interface using Streamlit

3.5 Use Case Diagram

The proposed document question answering system involves a single primary actor: the **User**, who interacts with the application by uploading PDF documents and asking questions.

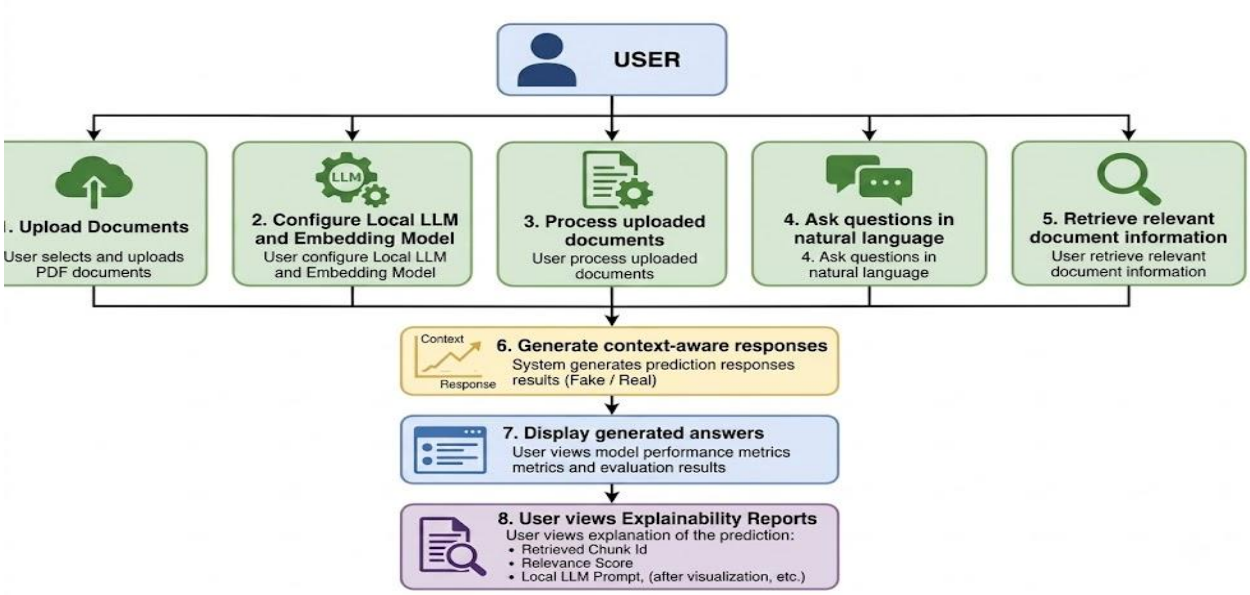


Figure 3.2 Use Case Model of the Proposed System

Use Case Description

- Upload PDF documents.
- Configure Local LLM and Embedding Model.
- Process uploaded documents.
- Ask questions in natural language.
- Retrieve relevant document information.
- Generate context-aware responses.
- Display generated answers.

3.6 Activity Diagram

The activity diagram illustrates the sequence of operations performed by the proposed Hybrid RAG framework

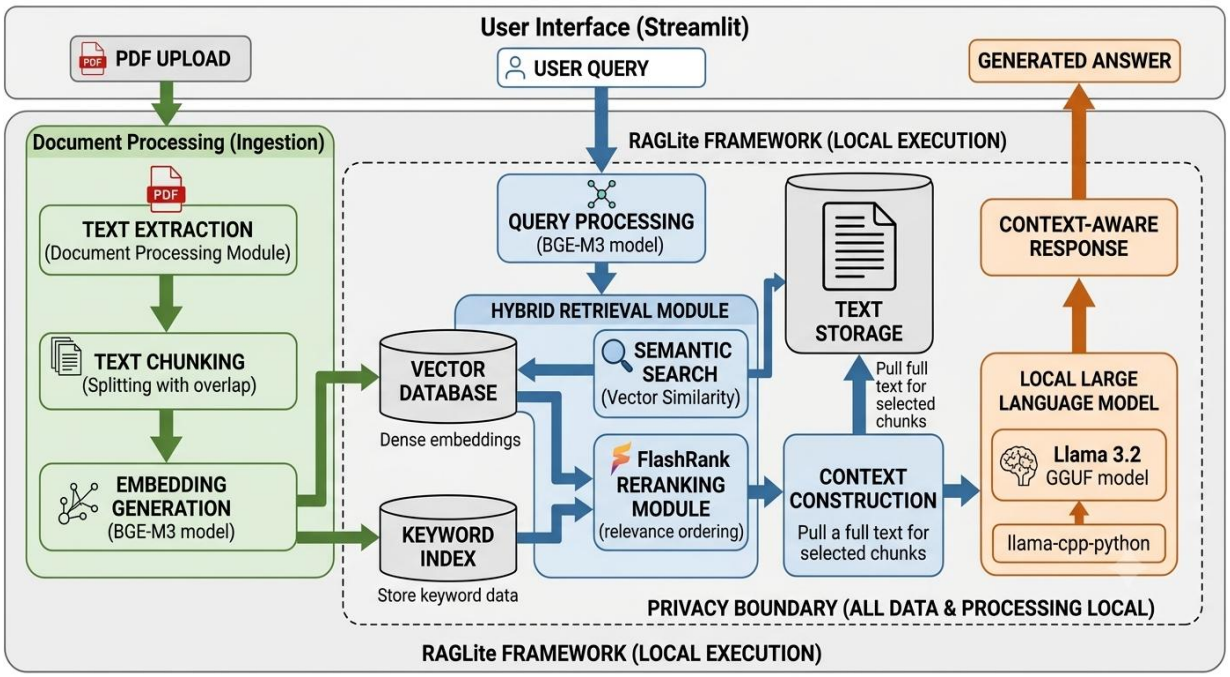


Figure 3.3 System Workflow Activity Diagram

3.7 Database Design

The proposed system primarily focuses on document retrieval and question answering using the RAGLite framework. Instead of storing structured application data in a traditional relational database, the system processes uploaded PDF documents, generates embeddings, and creates a retrieval index for efficient document search. The application also supports PostgreSQL as a configurable backend through RAGLite when database persistence is required.

Table 3.3 Dataset Schema

Field	Data Type	Description
Document ID	Integer	Unique identifier for each uploaded document
File Name	Text	Uploaded PDF file name
Document Text	Long Text	Extracted textual content
Upload Time	DateTime	Document upload timestamp

Feature Table

During preprocessing, the uploaded PDF is processed into text chunks. Semantic embeddings and retrieval-related information are generated for each chunk to support efficient document retrieval and question answering.

Table 3.4 Document Chunk and Embedding Information

Field	**Description**
Chunk ID	Unique identifier assigned to each text chunk
Chunk Text	Text extracted from the uploaded PDF
Embedding Vector	Dense vector representation generated using the BGE-M3 embedding model
Chunk Order	Position of the chunk within the original document
Source Document	Original PDF document from which the chunk was extracted

The generated document chunks, semantic embeddings, and retrieval information are combined within the Hybrid Retrieval-Augmented Generation pipeline to improve retrieval accuracy and generate context-aware responses.

CHAPTER IV

METHODOLOGY

The methodology adopted in this project follows a systematic Retrieval-Augmented Generation (RAG) pipeline for intelligent document question answering. The proposed framework integrates document processing, Natural Language Processing (NLP), semantic embeddings, Hybrid Search, reranking, and a Local Large Language Model (LLM) to generate accurate responses from uploaded PDF documents. The complete methodology consists of document upload, text extraction, chunking, embedding generation, Hybrid Search, reranking, context construction, and response generation. This structured workflow enables the system to retrieve relevant information efficiently while ensuring privacy through local inference.

4.1 Development Approach

The project follows a modular development approach in which each component of the Retrieval-Augmented Generation pipeline is implemented and validated before integrating it with the complete system. This methodology ensures reliable document processing, accurate retrieval, and context-aware response generation.

The overall development workflow consists of the following phases:

Step 1: PDF Document Upload

The first stage allows the user to upload one or more PDF documents through the Streamlit interface. The uploaded documents serve as the knowledge source for answering user queries.

The system performs:

- PDF selection
- File validation
- Document loading

Step 2: Text Extraction

After successful upload, the system extracts textual content from the PDF document.

The extraction process includes:

- Reading PDF pages
- Extracting raw text
- Removing unnecessary formatting
- Combining page contents

The extracted text becomes the input for subsequent processing.

Step 3: Text Chunking

Instead of processing the entire document at once, the extracted text is divided into smaller overlapping chunks.

Chunking provides several advantages:

- Preserves contextual information
- Improves retrieval accuracy
- Reduces LLM context size
- Supports efficient semantic search

Each chunk is assigned a unique identifier for retrieval.

Step 4: Embedding Generation

Each document chunk is converted into a dense vector representation using the BGE-M3 embedding model.

Unlike keyword search, embeddings represent the semantic meaning of the text.

The embedding generation stage includes:

- Encoding text chunks
- Generating dense vectors
- Associating embeddings with document chunks

These embeddings are later used during semantic retrieval.

Step 5: Local Document Indexing

After embeddings are generated, the document chunks and their corresponding embeddings are organized into a local retrieval index.

The retrieval index enables efficient searching without scanning the entire document repeatedly.

The indexing process includes:

- Organizing document chunks
- Storing embedding vectors
- Maintaining document metadata

Step 6: User Query Processing

The user enters a question through the Streamlit interface.

The system performs:

- Query preprocessing
- Query embedding generation
- Preparation for retrieval

The user query is transformed into the same embedding space as the document chunks.

Step 7: Hybrid Search

The proposed system performs **Hybrid Search**, which combines:

- Semantic Retrieval
- Keyword-Based Retrieval

Semantic retrieval identifies contextually similar document chunks, while keyword retrieval matches exact technical terms.

Combining both techniques improves retrieval precision.

Step 8: FlashRank Reranking

The retrieved document chunks are further processed using **FlashRank**.

FlashRank:

- Calculates relevance scores
- Removes less relevant chunks
- Reorders search results

Only the highest-ranked document chunks are selected.

Step 9: Context Construction

The selected document chunks are combined into a single context window.

This context provides the Local Large Language Model with relevant document information before response generation.

Step 10: Local LLM Response Generation

The constructed context and user query are provided to the Llama 3.2 Large Language Model, executed locally through llama-cpp-python.

The model generates a context-aware answer based only on the retrieved document information.

Since inference is performed locally, no document data is transmitted to external servers.

4.2 Algorithms / Techniques Used

The proposed Hybrid RAG framework combines multiple modern AI techniques to improve document retrieval and question answering.

4.2.1 PDF Text Extraction

he uploaded PDF documents are processed to extract textual information.

This stage includes:

- Reading PDF pages
- Extracting text

- Removing unnecessary formatting
- Preparing text for chunking

4.2.2 Text Chunking

The extracted document is divided into overlapping text chunks.

Advantages include:

- Better context preservation
- Improved retrieval performance
- Efficient LLM processing
- Reduced context overflow

4.2.3 Embedding Generation (BGE-M3)

The BGE-M3 embedding model converts text into dense vector representations.

Key characteristics include:

- Semantic representation
- High retrieval accuracy
- Multilingual capability
- Efficient vector search

Embeddings enable the system to retrieve information based on meaning rather than exact words.

4.2.4 Hybrid Search

Hybrid Search combines:

- Semantic Search
- Keyword Search

Advantages include:

- Improved retrieval precision
- Better handling of technical documents
- Context-aware document matching
- Exact keyword support

4.2.5 FlashRank Reranking

FlashRank improves retrieval quality by reordering retrieved document chunks according to their relevance.

Advantages include:

- Higher retrieval accuracy
- Better context selection
- Reduced irrelevant information
- Improved LLM response quality

4.3 Tools and Technologies

The implementation of the proposed Hybrid RAG framework utilizes various software tools, programming libraries, and hardware resources.

Table 4.1 Software Requirements

Software	Purpose
Python	Backend programming language
Visual Studio Code	Source code editor
Streamlit	User Interface
Git	Version Control
GitHub	Project Repository

Table 4.2 Python Libraries

Library	Purpose
RAGLite	**Document processing and RAG pipeline**
Streamlit	**Interactive web interface**
llama-cpp-python	**Local LLM inference**
FlashRank	**Reranking retrieved chunks**

Sentence Transformers / BGE-M3	**Embedding generation**
PyMuPDF / pypdf	**PDF text extraction**
NumPy	**Numerical computations**
Pandas	**Data handling**

Table 4.3 Hardware Requirements

Component	Specification
Processor	**AMD Ryzen 7 / Intel Core i5 or above**
RAM	**16 GB or higher**
Storage	**10 GB free space**
GPU	**NVIDIA RTX 4050 (Optional for acceleration)**
Operating System	**Windows 11**

Table 4.4 Technologies Used in the Project

Technology	**Application**
Retrieval-Augmented Generation (RAG)	Document Question Answering
Hybrid Search	Semantic + Keyword Retrieval
BGE-M3	Embedding Generation
FlashRank	Retrieval Reranking
Llama 3.2	Local Response Generation
GGUF	Optimized Local Model Format
llama-cpp-python	Local Model Inference
Streamlit	User Interface
RAGLite	Complete RAG Pipeline

CHAPTER V

IMPLEMENTATION

5.1 Module Description

The proposed Hybrid Retrieval-Augmented Generation (RAG) system is divided into several independent modules. Each module performs a specific function within the overall document question answering pipeline.

Module 1: PDF Upload Module

Purpose: This module allows users to upload one or more PDF documents through the Streamlit interface. The uploaded documents serve as the knowledge source for the question answering system.

Major Functions:

- Upload PDF documents
- Validate uploaded files
- Read document information
- Store uploaded files temporarily
- Prepare documents for processing

Output: Uploaded PDF documents ready for text extraction.

Module 2: Document Processing Module

Purpose : This module extracts textual information from uploaded PDF documents and prepares it for further processing.

Operations Performed:

- Read PDF pages
- Extract textual content
- Remove unnecessary formatting
- Combine extracted text
- Prepare clean document text

Output: Structured document text ready for chunking.

Module 3: Text Chunking Module

Purpose: This module divides the extracted document into smaller overlapping text chunks.

Operations Performed:

- Split document into chunks
- Maintain chunk sequence
- Preserve contextual continuity

- Generate chunk identifiers

Output : Collection of document chunks for embedding generation.

Module 4: Embedding Generation Module

Purpose: This module converts each document chunk into dense semantic embeddings using the **BGE-M3 embedding model**.

Functions:

- Generate vector embeddings
- Associate embeddings with chunks
- Prepare semantic representations

Output: Vector embeddings representing document semantics.

Module 5: Hybrid Retrieval Module

Purpose: This module retrieves relevant document chunks by combining semantic retrieval and keyword-based retrieval.

Components:

- Semantic similarity search
- Keyword matching
- Retrieval score calculation
- Candidate chunk selection

Output: Relevant document chunks retrieved from the uploaded document.

Module 6: FlashRank Reranking Module

Purpose: This module improves retrieval quality by reranking the retrieved document chunks according to their relevance.

Functions:

- Score retrieved chunks
- Remove less relevant chunks
- Select top-ranked context

Output: Optimized document context for response generation.

Module 7: Local LLM Response Generation Module

Purpose: This module generates context-aware responses using the Llama 3.2 Large Language Model running locally through llama-cpp-python.

Components:

- Load GGUF model
- Accept user query
- Process retrieved context
- Generate natural language response

Output: Final answer generated from the uploaded document.

Module 8: User Interface Module

Purpose: Provides an interactive interface developed using Streamlit.

Functions:

- Upload PDF documents
- Ask questions
- Display generated responses
- Show processing status

Output: Interactive document question answering application.

5.2 User Interface Design

Although the project is implemented primarily within a Jupyter Notebook/Google Colab environment, the workflow is organized to provide a clear and user-friendly interaction for data analysis, model training, prediction, and visualization.

- **User Interface Objectives:** Provides an interface for loading the dataset, visualizing preprocessing and analysis results, monitoring model training, evaluating performance, and displaying prediction explanations.
- **Input Screen:** The initial stage allows the user to load the dataset and inspect its structure.
- **Model Training Interface:** The notebook displays model training progress using epoch-wise statistics.

5.3 Coding and Integration

The proposed system follows a modular implementation approach in which each component is developed independently and integrated into a unified Retrieval-Augmented Generation pipeline.

Step 1: Importing Libraries

The implementation begins by importing the required libraries for document processing, embedding generation, retrieval, local inference, and user interface development.

Examples include:

- Streamlit
- RAGLite

- llama-cpp-python
- FlashRank
- Sentence Transformers
- NumPy
- Pandas

Step 2: PDF Upload

The uploaded PDF document is received through the Streamlit interface.

Operations include:

- File validation
- PDF loading
- Temporary storage

Step 3: Text Extraction

The system extracts textual information from the uploaded PDF.

Operations include:

- Reading document pages
- Extracting text
- Cleaning extracted content

Step 4: Text Chunking

The extracted text is divided into smaller overlapping chunks.

The chunking process:

- Preserves document context
- Improves retrieval efficiency
- Supports semantic search

Step 5: Embedding Generation

Each text chunk is converted into a semantic vector using the BGE-M3 embedding model.

The generated embeddings represent the contextual meaning of document content and are used during retrieval.

Step 6: Hybrid Search

The user enters a question through the interface.

The system performs:

- Query embedding generation
- Semantic similarity search
- Keyword matching

Both retrieval methods are combined to identify the most relevant document chunks.

Step 7: FlashRank Reranking

The retrieved document chunks are reranked according to their relevance.

The highest-ranked chunks are selected to construct the context provided to the Local Large Language Model.

Step 8: Local Response Generation

The selected context and user query are provided to the Llama 3.2 GGUF model through llama-cpp-python.

The Local Large Language Model generates an accurate and context-aware response based solely on the retrieved document information.

Step 9: Display Response

Finally, the generated response is displayed through the Streamlit interface, allowing users to interact with the uploaded document in natural language.

5.4 Screenshots and Outputs

The implementation generates multiple outputs throughout the document processing and question answering pipeline. The following screenshots illustrate the major stages of the proposed Hybrid Retrieval-Augmented Generation system.

5.4.1 Home Page / Application Interface

Purpose: Displays the main Streamlit interface where users can configure models and interact with the document question answering system.

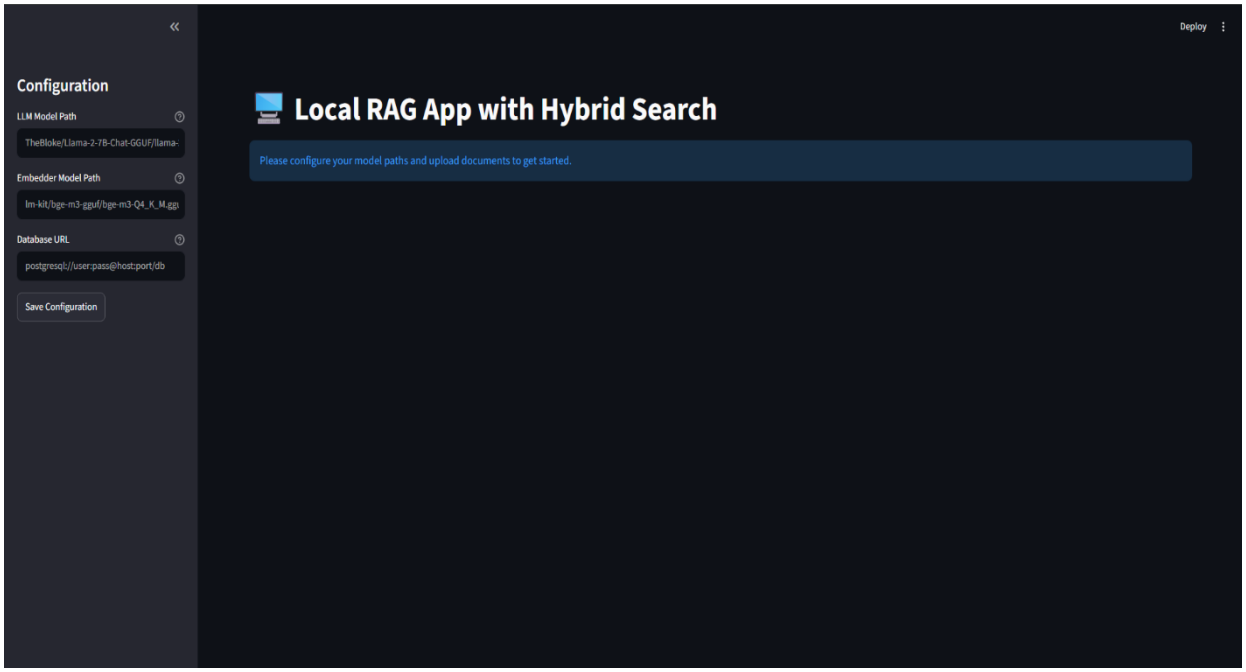


Figure 5.1 Home Page of the Hybrid RAG Application

5.4.2: PDF Upload Interface

Purpose: Shows the interface for uploading one or more PDF documents into the system before processing.

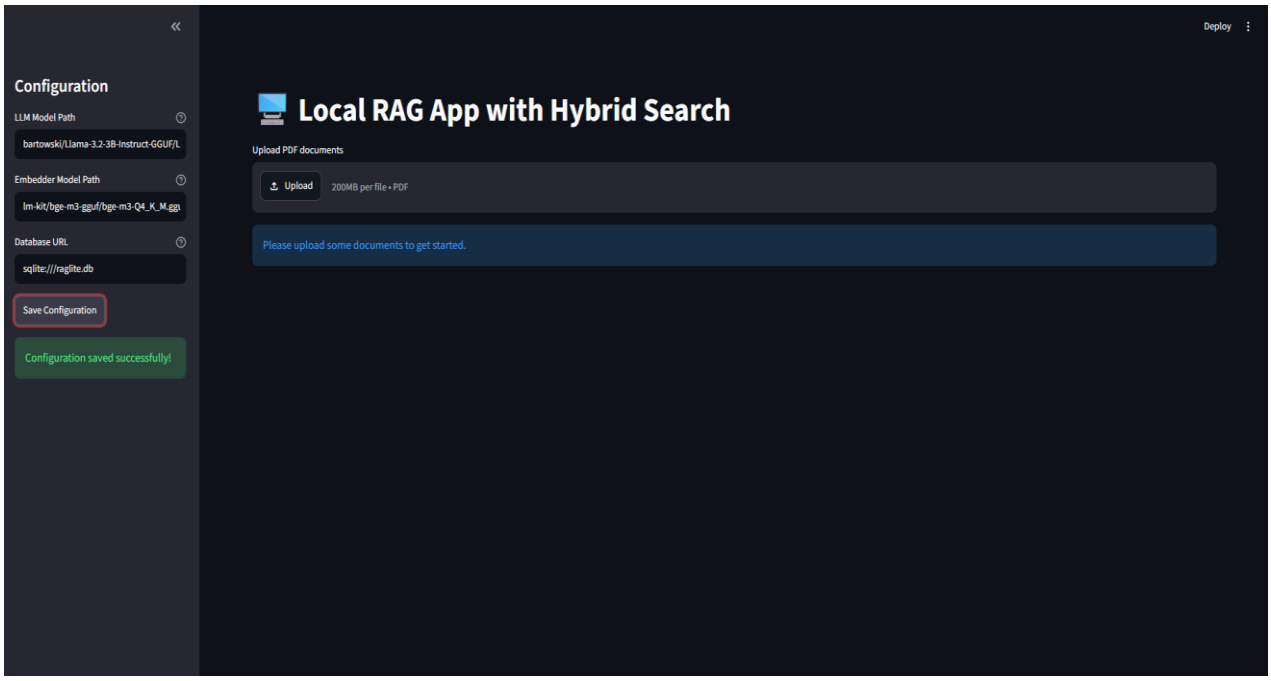


Figure 5.2 PDF Upload Interface

5.4.3: Document Processing Output

Purpose: Displays successful document processing, including text extraction, chunking, and embedding generation.



Figure 5.3 Document Processing Status

5.4.4: Model Configuration

Purpose: Displays the configuration of the Local Large Language Model, embedding model, and retrieval settings used by the application.

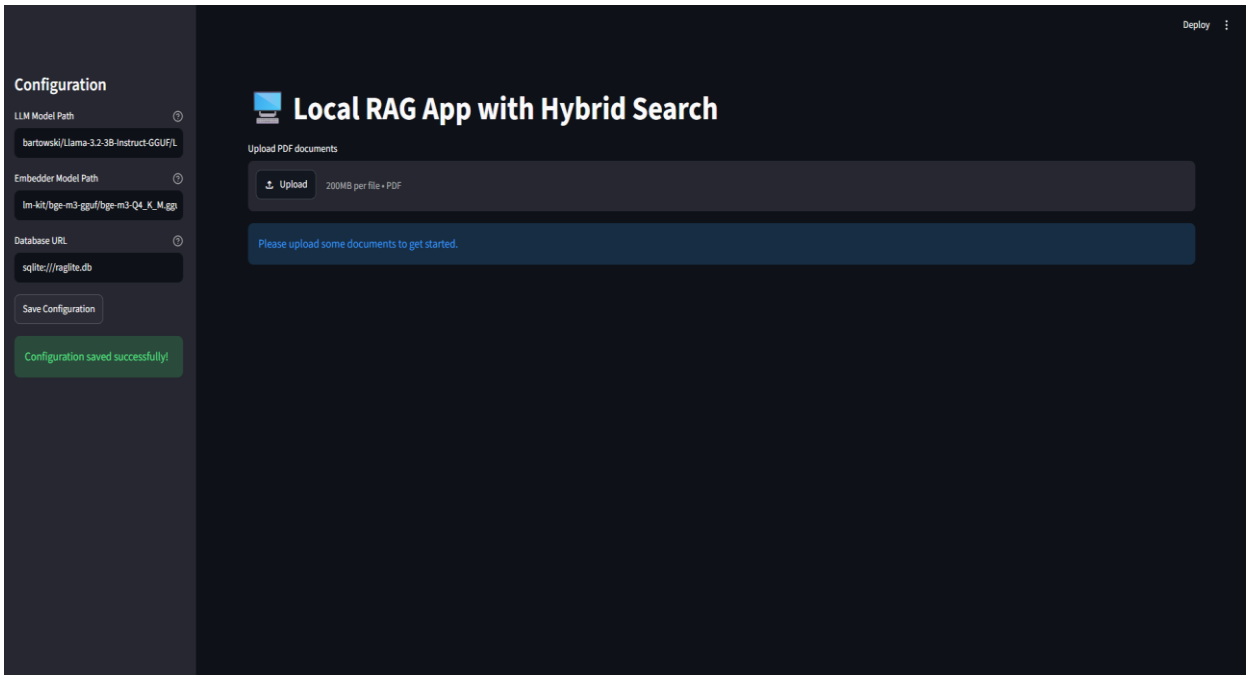


Figure 5.4 Model Configuration Screen

5.4.5: User Question Interface

Purpose : Illustrates the interface where users enter natural language questions related to the uploaded document.

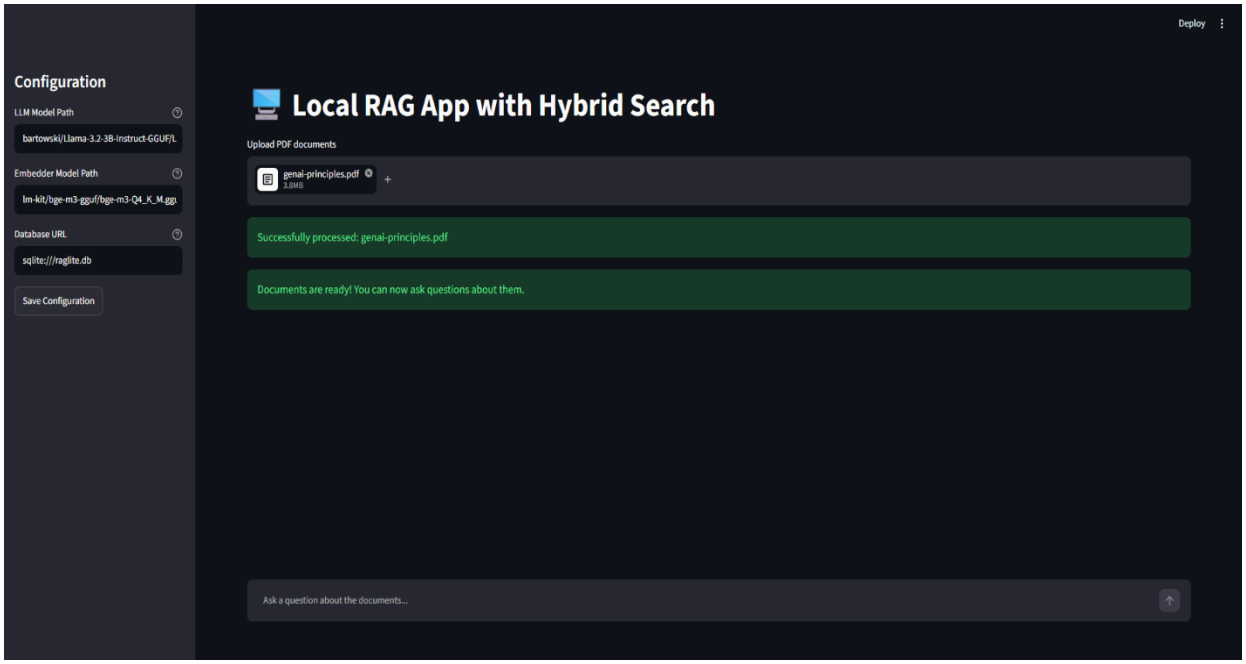


Figure 5.5 Question Input Interface

5.4.6: Generated Response

Purpose: Displays the context-aware response generated by the Local Llama 3.2 model based on the uploaded PDF document.

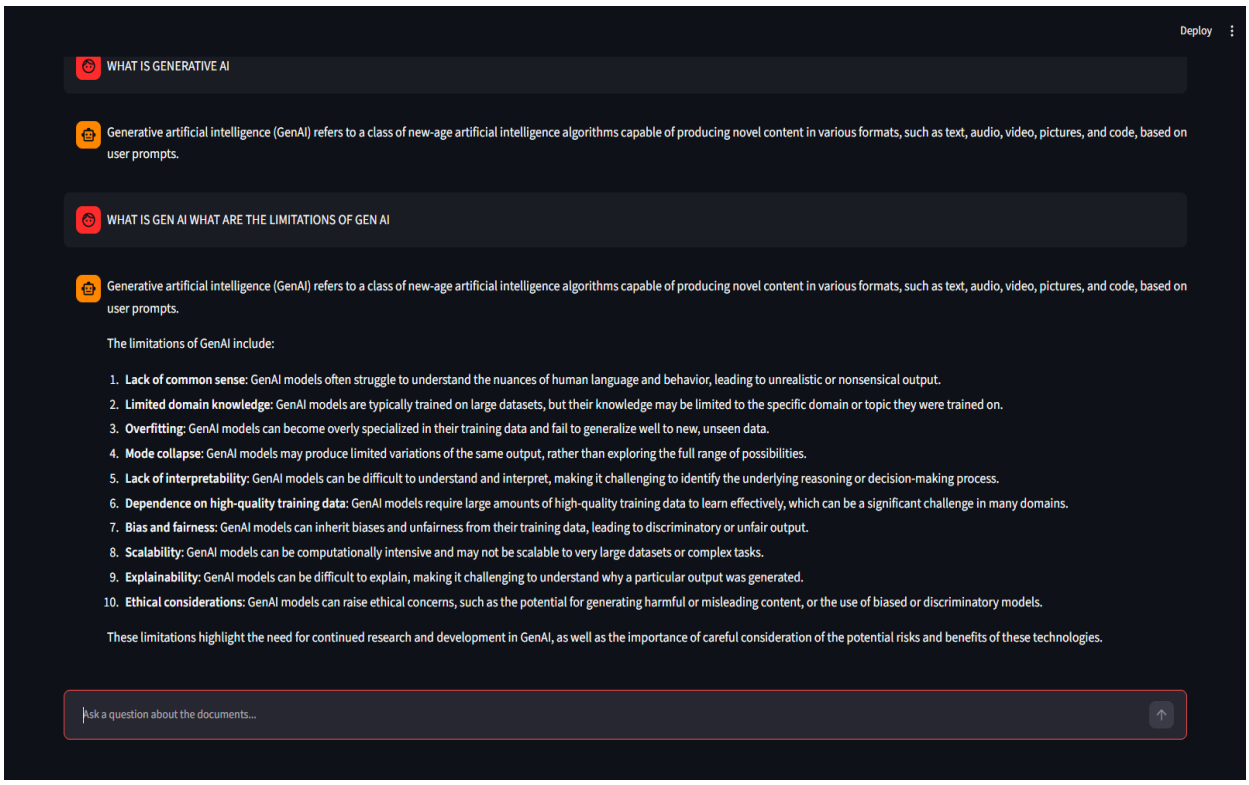


Figure 5.6 Generated Response

CHAPTER VI

RESULTS AND DISCUSSION

6.1 Experimental Results

The proposed Hybrid Retrieval-Augmented Generation (RAG) system was implemented and tested using various PDF documents to evaluate its document processing, retrieval, and question answering capabilities. The experiments were conducted after integrating document processing, embedding generation, Hybrid Search, FlashRank reranking, and the Local Large Language Model into a unified application.

The developed system successfully processes uploaded PDF documents, extracts textual information, generates semantic embeddings, retrieves relevant document chunks, and produces context-aware responses using the Local Llama 3.2 model. The complete workflow was executed through the Streamlit interface, allowing users to upload documents and interact with the system using natural language queries.

The experimental results demonstrate that the proposed Hybrid RAG framework effectively retrieves relevant information from uploaded documents and generates meaningful responses while preserving user privacy through local inference.

Document Processing Results

After uploading a PDF document, the system successfully performs the following operations:

- PDF document upload
- Text extraction
- Document chunking
- Embedding generation
- Local retrieval indexing

These preprocessing stages prepare the uploaded document for efficient Hybrid Search and response generation.

Retrieval Results

The Hybrid Search mechanism combines semantic retrieval with keyword-based retrieval to identify the most relevant document chunks.

The retrieval process successfully:

- Retrieves contextually relevant document sections
- Matches important keywords
- Improves retrieval accuracy

- Provides relevant context for response generation

The addition of FlashRank further improves retrieval quality by reranking retrieved document chunks according to their relevance.

Response Generation Results

The Local Large Language Model successfully generates responses using the retrieved document context.

The generated responses are:

- Context-aware
- Generated in natural language
- Produced without cloud-based AI services

The system allows users to continue asking multiple questions from the same uploaded document without reprocessing the PDF.

6.2 System Performance Analysis

The performance of the proposed system is evaluated based on its document processing capability, retrieval efficiency, response quality, and usability.

The evaluation focuses on the following parameters:

- Document processing
- Embedding generation
- Hybrid Search
- Retrieval relevance
- Response generation
- Local inference

Table 6.1 System Performance Evaluation

Parameter	Observation
PDF Upload	Successfully uploads PDF documents
Text Extraction	Correctly extracts document content
Text Chunking	Successfully divides documents into chunks
Embedding Generation	Generates semantic embeddings using BGE-M3
Hybrid Search	Retrieves relevant document chunks
FlashRank	Improves ranking of retrieved results
Local LLM	Generates context-aware responses
User Interface	Interactive and easy to use

Response Quality Analysis

The quality of generated responses depends on the relevance of the retrieved document chunks. The proposed Hybrid Search approach improves response quality by combining semantic similarity with keyword matching. FlashRank further enhances the retrieved context before it is provided to the Local Large Language Model.

Since the response generation is grounded in retrieved document information, the system reduces the possibility of hallucinated responses compared to standalone Large Language Models.

Local Inference Analysis

Unlike cloud-based AI services, the proposed system performs inference locally using the Llama 3.2 GGUF model.

Local inference provides several benefits:

- Improved privacy
- Offline accessibility
- Reduced dependency on external APIs
- Better control over sensitive documents

6.3 Discussion of Results

The implementation results demonstrate that the proposed Hybrid Retrieval-Augmented Generation framework successfully integrates document processing, semantic embeddings, Hybrid Search, reranking, and Local Large Language Models into a unified document question answering system.

The BGE-M3 embedding model effectively captures the semantic meaning of document text, while Hybrid Search improves retrieval by combining semantic similarity and keyword matching. FlashRank enhances retrieval relevance by selecting the most informative document chunks before passing them to the Local LLM.

The Streamlit-based user interface provides an interactive environment for uploading PDF documents and asking questions in natural language. The Local Llama 3.2 model generates responses using the retrieved document context, ensuring accurate and privacy-preserving document question answering.

Overall, the proposed system demonstrates the practical applicability of Retrieval-Augmented Generation for intelligent document understanding while supporting offline operation through local inference.

Advantages of the Proposed System

- Supports intelligent document question answering.
- Combines semantic retrieval and keyword-based retrieval.
- Improves retrieval relevance using FlashRank.
- Generates context-aware responses using a Local Large Language Model.
- Preserves user privacy through local inference.
- Supports offline document processing.
- Provides an interactive user interface using Streamlit.
- Modular architecture suitable for future enhancements.

Limitations

Based on the current implementation, the following limitations can be identified:

- The system currently supports PDF documents only.
- Response quality depends on the quality of the uploaded document.
- Large PDF collections may increase retrieval time.
- Scanned PDF documents require OCR support for text extraction.
- The current implementation uses a single Local Large Language Model configuration.
- Hardware resources influence local inference performance.
- Performance evaluation is primarily functional; comprehensive benchmarking across multiple datasets remains future work.

CHAPTER VII

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The rapid growth of digital documents in organizations, educational institutions, and industries has increased the need for intelligent document retrieval and question answering systems. Traditional keyword-based search methods often fail to understand the contextual meaning of user queries, making information retrieval difficult when dealing with large collections of documents. Although cloud-based Large Language Models provide advanced language understanding capabilities, they introduce privacy concerns because sensitive documents are processed on external servers. To overcome these limitations, this project proposed a Hybrid Retrieval-Augmented Generation (RAG) System for Intelligent Document Question Answering using Local Large Language Models.

The implementation begins with PDF document upload and text extraction, followed by document chunking and semantic embedding generation using the BGE-M3 embedding model. These embeddings represent the contextual meaning of document content and enable efficient semantic retrieval. The proposed system combines semantic retrieval with keyword-based retrieval through Hybrid Search, improving the relevance of retrieved document chunks compared to traditional retrieval techniques.

To further enhance retrieval quality, the project integrates FlashRank for reranking the retrieved document chunks before they are provided as context to the Llama 3.2 Large Language Model. The Local LLM, executed using llama-cpp-python, generates accurate and context-aware responses based solely on the retrieved document information. Since the complete pipeline operates locally, the system provides improved privacy, offline accessibility, and reduced dependency on cloud-based AI services.

The experimental implementation demonstrates that the proposed framework successfully processes uploaded PDF documents, retrieves relevant information using Hybrid Search, and generates meaningful responses through Local LLM inference. The Streamlit-based user interface provides an interactive platform for uploading documents and asking questions in natural language. The integration of RAGLite, semantic embeddings, FlashRank, and Local LLMs provides an efficient and scalable solution for intelligent document question answering.

Overall, the objectives of the project have been successfully achieved. The developed Hybrid Retrieval-Augmented Generation framework provides an accurate, privacy-preserving, and user-

friendly solution for document-based question answering. It demonstrates the effectiveness of combining semantic retrieval, keyword-based retrieval, reranking, and Local Large Language Models to improve intelligent information retrieval from PDF documents.

7.2 Future Enhancements

Although the proposed Hybrid RAG framework demonstrates effective document retrieval and question answering capabilities, several enhancements can further improve its performance and applicability.

1. Support for Multiple Document Formats

The current implementation primarily supports PDF documents. Future work can extend the framework to process additional document formats such as Microsoft Word documents, PowerPoint presentations, Excel spreadsheets, HTML pages, and plain text files.

2. OCR-Based Document Processing

Future implementations can integrate Optical Character Recognition (OCR) to process scanned PDF documents and image-based files. This enhancement would enable the system to retrieve information from documents that do not contain machine-readable text.

3. Voice-Based Interaction

The system can be extended with speech recognition and text-to-speech technologies, allowing users to ask questions using voice commands and receive spoken responses, thereby improving accessibility and user experience.

4. Advanced Large Language Models

Future versions of the project can evaluate and integrate more advanced Local Large Language Models and embedding models to improve response quality, retrieval accuracy, and reasoning capabilities while maintaining offline operation.

5. Scalable Database and Enterprise Deployment

The framework can be enhanced by integrating scalable vector databases and enterprise document management systems to efficiently handle large document collections. Cloud and hybrid deployment options can also be explored to support collaborative environments while maintaining secure document retrieval and question answering.

REFERENCES

[1] Lewis, P., Perez, E., Piktus, A., et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[2] Touvron, H., Lavril, T., Izacard, G., et al., "Llama 3: Open Foundation and Instruction Models," Meta AI, 2024.

[3] Xiao, S., Liu, Z., Zhang, P., and Muennighoff, N., "BGE M3-Embedding: Multi-Functionality, Multi-Linguality, and Multi-Granularity Text Embeddings," *arXiv preprint* arXiv:2402.03216, 2024.

[4] Robertson, S. E., and Zaragoza, H., "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[5] Manning, C. D., Raghavan, P., and Schütze, H., *Introduction to Information Retrieval*, Cambridge University Press, 2008.

[6] Reimers, N., and Gurevych, I., "Sentence-BERT: Sentence Embeddings using Siamese BERT Networks," *Proceedings of EMNLP-IJCNLP*, pp. 3982–3992, 2019.

[7] Karpukhin, V., Oguz, B., Min, S., et al., "Dense Passage Retrieval for Open-Domain Question Answering," *Proceedings of EMNLP*, pp. 6769–6781, 2020.

[8] Vaswani, A., Shazeer, N., Parmar, N., et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[9] Brown, T. B., Mann, B., Ryder, N., et al., "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.

[10] Devlin, J., Chang, M. W., Lee, K., and Toutanova, K., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *Proceedings of NAACL-HLT*, pp. 4171–4186, 2019.

APPENDICES

The appendices provide supplementary information that supports the implementation and evaluation of the proposed Hybrid Deep Learning Framework for Fake News Detection. This section includes additional details such as software and hardware requirements, project structure, important source code snippets, sample prediction outputs, experimental results, libraries used, and abbreviations. These materials complement the main report by providing technical references and supporting evidence without interrupting the flow of the core discussion. The appendices serve as a valuable resource for understanding the implementation details and reproducing the proposed system. Below is the Appendices section written in paragraph form, suitable for inclusion in your mini project report.

1. Appendix A: Software and Hardware Requirements

The proposed Hybrid Retrieval-Augmented Generation system was developed using Python 3.10 and Visual Studio Code as the primary development environment. The application utilizes the Streamlit framework to provide an interactive web interface. The implementation relies on several open-source libraries, including RAGLite, llama-cpp-python, FlashRank, Sentence Transformers (BGE-M3), PyPDF, NumPy, and Pandas, which support document processing, embedding generation, retrieval, reranking, and Local Large Language Model inference. The system was executed on a computer with an AMD Ryzen 7 processor, 16 GB RAM, NVIDIA RTX 4050 Laptop GPU, and Windows 11 operating system. The implementation primarily performs local inference and therefore does not require cloud-based AI services.

2. Appendix B: Project Directory Structure

The project is organized into multiple files and folders to maintain a clear and modular architecture. The application source code is contained in the main Python files responsible for the Streamlit interface and Retrieval-Augmented Generation pipeline. The project includes configuration files for Local Large Language Models and embedding models, while uploaded PDF documents are processed during runtime. The project also contains a requirements.txt file listing all Python dependencies required for execution. Additional resources include the project README, model configuration files, and supporting scripts used for document processing and retrieval.

3. Appendix C: Source Code Snippets

The implementation begins by importing essential Python libraries such as Streamlit**,** RAGLite**,** llama-cpp-python, FlashRank, Sentence Transformers, and PyPDF**.** After the application starts, users upload PDF documents through the Streamlit interface. The uploaded documents are processed using the RAGLite framework, which extracts text, divides the

document into smaller chunks, generates semantic embeddings using the BGE-M3 embedding model, and prepares a retrieval index. When a user submits a question, the system performs Hybrid Search by combining semantic retrieval and keyword matching. The retrieved document chunks are reranked using FlashRank before being supplied as context to the Llama 3.2 Large Language Model, executed locally through llama-cpp-python, to generate the final response.

4. Appendix D: Sample Question Answering Output

The developed application allows users to upload PDF documents and ask questions in natural language. The system retrieves relevant information from the uploaded document and generates accurate, context-aware responses using the Local Large Language Model. Multiple questions can be asked from the same document without re-uploading the file, demonstrating the effectiveness of the Hybrid Retrieval-Augmented Generation framework for intelligent document understanding.

5. Appendix E: Experimental Results

Several outputs were generated during system implementation to verify the successful execution of the Retrieval-Augmented Generation pipeline. These include the Streamlit home interface, PDF upload screen, document processing status, embedding generation, Hybrid Search execution, FlashRank reranking, and Local LLM response generation. Sample question-answering outputs demonstrate that the proposed system successfully retrieves relevant document information and generates meaningful responses using only the uploaded PDF documents.

6. Appendix F: Libraries Used

The implementation utilizes several Python libraries to perform document processing, retrieval, and response generation. Streamlit provides the interactive user interface, while RAGLite manages the Retrieval-Augmented Generation pipeline. llama-cpp-python enables local execution of the Llama 3.2 GGUF model for response generation. The BGE-M3 embedding model is used to generate semantic vector representations of document text, and FlashRank improves retrieval quality through reranking. PyPDF supports PDF text extraction, while NumPy and Pandas assist in numerical computation and data handling throughout the application.

7. Appendix G: Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
GGUF	GPT-Generated Unified Format
UI	User Interface
PDF	Portable Document Format
BGE	Beijing General Embedding Model
OCR	Optical Character Recognition
API	Application Programming Interface
GPU	Graphics Processing Unit
CPU	Central Processing Unit
RAGLite	Retrieval-Augmented Generation Lite Framework

Source Code Snippets

Snippet 1 : Importing Required Libraries

This snippet imports the required libraries for the Streamlit user interface, the RAGLite framework, and the FlashRank reranker used in the Hybrid RAG pipeline.

```
import os
import logging
import streamlit as st
from raglite import RAGLiteConfig, insert_document, hybrid_search, retrieve_chunks, rerank_chunks, rag
from rerankers import Reranker
```

Snippet 2 : RAGLite Configuration

This code initializes the RAGLite configuration by specifying the database, Local LLM, embedding model, chunk size, and FlashRank reranker.

```
return RAGLiteConfig(
    db_url=settings["DBUrl"],
    llm=f"llama-cpp-python/{settings['LLMPath']}",
    embedder=f"llama-cpp-python/{settings['EmbedderPath']}",
    embedder_normalize=True,
    chunk_max_size=512,
    reranker=Reranker("ms-marco-MiniLM-L-12-v2", model_type="flashrank")
)
```

Snippet 3: Document Processing

This function processes the uploaded PDF document by extracting text, splitting it into chunks, generating embeddings, and preparing it for retrieval.

```
    insert_document(
        Path(file_path),
        config=st.session_state.my_config)
```

Snippet 4: Hybrid Search

This snippet performs Hybrid Search by combining semantic retrieval and keyword search to identify the most relevant document chunks.

```
chunk_ids, scores = hybrid_search(query, num_results=10, config=st.session_state.my_config)
```

Snippet 5: Retrieving and Reranking Chunks

The retrieved document chunks are fetched and reranked using FlashRank to improve retrieval quality before being sent to the Local LLM.

```
chunks = retrieve_chunks(chunk_ids, config=st.session_state.my_config)
return rerank_chunks(query, chunks, config=st.session_state.my_config)
```

Snippet 6 : Fallback Answer Generation

If no relevant document chunks are retrieved, the system uses the Local LLM's general knowledge to generate a response.

```
response_stream = rag(
    prompt=query,
    system_prompt=system_prompt,
    search=None,
    messages=[],
    max_tokens=1024,
    temperature=0.7,
    config=st.session_state.my_config
)
```

Snippet 7 : PDF Upload Interface

This code creates the Streamlit interface that allows users to upload one or more PDF documents for processing.

```
uploaded_files = st.file_uploader(
    "Upload PDF documents",
    type=["pdf"],
    accept_multiple_files=True,
    key="pdf_uploader"
)
```

Snippet 8 : User Question Input

This snippet accepts questions from the user and forwards them to the Hybrid RAG pipeline for document retrieval and answer generation.

```
user_input = st.chat_input("Ask a question about the documents...")
```

Snippet 9 : RAG Answer Generation

This function generates the final response by supplying the retrieved document context to the Local Llama 3.2model through the RAGLite framework.

```
response_stream = rag(
    prompt=user_input,
    system_prompt=RAG_SYSTEM_PROMPT,
    search=hybrid_search,
    messages=formatted_messages,
    max_contexts=5,
    config=st.session_state.my_config
)
```

Snippet 10 : Application Entry Point

This is the entry point of the application that launches the Streamlit interface and starts the complete Hybrid RAG workflow

```
if __name__ == "__main__":
    main()
```